KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous), Coimbatore – 641 407 · Department of CSE

U21CSP12 – JavaScript Frameworks

ASSIGNMENT – II

Detail	
Name	Vigasvel R
Roll Number	24CS241
Year / Semester	III / V – B.E. CSE
Academic Year	2026 – 2027
Issue / Submission Date	15-10-2026 / 31-10-2026
GitHub Repository	https://github.com/_____

Questions Answered

Q.No	Task	Application	Stack	CO	Marks
Q1	Student Management System using MERN Stack	StudentHub	MERN	CO3	8
Q2 (Task 2)	Task management application for students	TaskPilot	MERN	CO5	16
Q3 (Task 2)	Appointment booking application	MediSlot	MEAN	CO4	16

Technologies

MongoDB + Mongoose (database), Express.js + Node.js (REST API), React 18 with Vite (MERN – Q1, Q2), Angular 20 with Reactive Forms (MEAN – Q3). Each app follows Browser → React/Angular → REST/JSON → Express → Mongoose → MongoDB. Complete source code and run steps are in the GitHub repository / zip.

Signature of the Student **Signature of the Course In-charge**

Q1 – StudentHub – Student Management System

Task: *Develop a Student Management System using MERN Stack: React form to enter student details, REST APIs with Express.js and Node.js, student records stored in MongoDB, add / view / update / delete operations.*

Stack: React 18 (Vite) + Axios · Node.js + Express.js · MongoDB (vigasvel_studenthub)

Requirements and Implementation

Requirement	Implementation
React form to enter student details	StudentModal.jsx – modal form with text, radio, select, range slider and checkbox inputs
REST APIs using Express.js and Node.js	routes/student.routes.js with validation middleware and central error handler
Store student records in MongoDB	models/student.model.js – Mongoose schema incl. attendance and virtual "eligible" field
Add, view, update and delete operations	Add / Edit in the modal, View with search & filters + dashboard, Delete with confirmation

Database

Collection fields: regNo (String), fullName (String), gender (String), email (String), department / section (String), attendance (Number), guardianName / guardianPhone (String), hosteller (Boolean), eligible (virtual).

REST API

Method	Endpoint	Purpose
GET	`/api/students`	List students (query: q, department, section, shortage=true)
GET	`/api/students/stats`	Dashboard numbers: total, average attendance, shortage, by department
GET	`/api/students/:id`	Get one student
POST	`/api/students`	Add a student (validation middleware, 422 / 409 errors)
PUT	`/api/students/:id`	Update a student
DELETE	`/api/students/:id`	Delete a student

Output

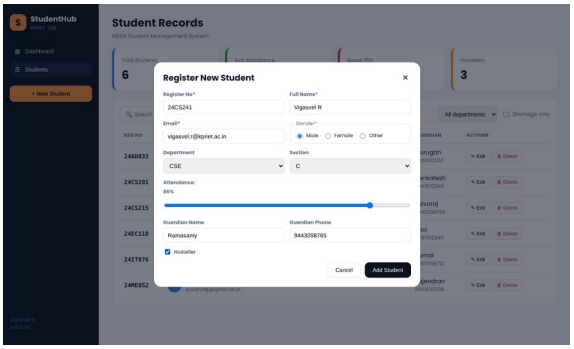


Fig. 1: CREATE – React form (modal) to enter student details

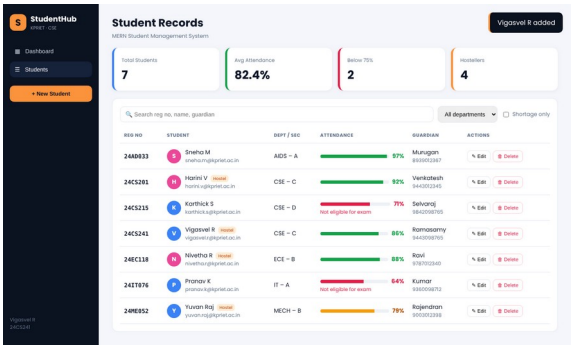


Fig. 2: New student added

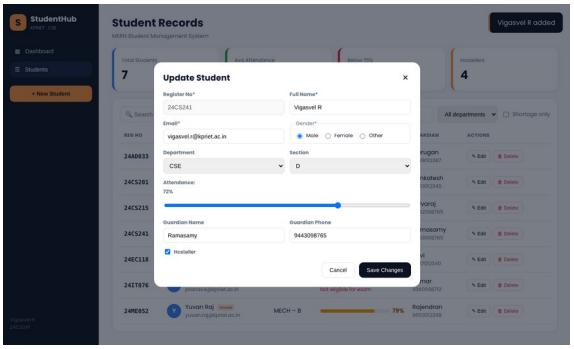


Fig. 3: UPDATE – editing attendance and section

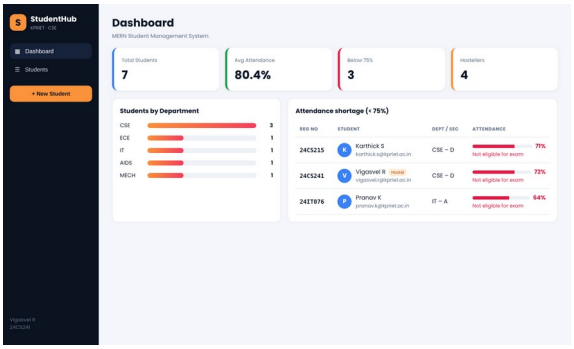


Fig. 4: Dashboard with statistics

Result: The application was implemented and all required operations were tested successfully.

Q2 (Task 2) – TaskPilot – Student Task Manager

Task: *Build a task management application for students: add, edit and delete tasks, set task priority and completion status, use React Hooks to manage the interface, store tasks in MongoDB through a Node.js backend.*

Stack: React 18 (Vite) + Axios – Hooks: useState, useEffect, useReducer, useMemo, useCallback, useRef, custom useTasks · Node.js + Express.js · MongoDB (vigasvel_taskpilot)

Requirements and Implementation

Requirement	Implementation
Add, edit and delete tasks	TaskForm.jsx for add/edit, Delete button on each card, POST / PUT / DELETE /api/tasks
Set task priority and completion status	Priority Low/Medium/High selector; status Pending / In Progress / Completed via board and checkbox
Use React Hooks to manage the interface	useState, useEffect, useReducer, useMemo, useCallback, useRef and custom hook useTasks()
Store tasks in MongoDB through a Node.js backend	Express API on port 5012 with Mongoose model task.model.js

Database

Collection fields: title (String), description (String), subject (String), priority (String), status (String), dueDate (Date), completedAt (Date).

REST API

Method	Endpoint	Purpose
GET	`/api/tasks`	List tasks sorted by priority then due date (query: status, priority, subject, q)
GET	`/api/tasks/subjects`	Distinct subjects
POST	`/api/tasks`	Add a task
PUT	`/api/tasks/:id`	Edit a task
PATCH	`/api/tasks/:id/status`	Change completion status
DELETE	`/api/tasks/:id`	Delete a task
DELETE	`/api/tasks?status=Completed`	Clear all completed tasks

Output

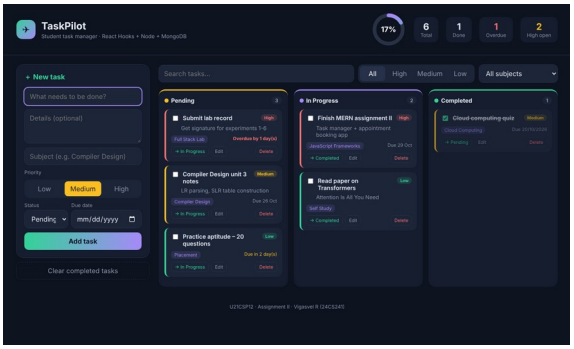


Fig. 5: Task board – tasks loaded from MongoDB

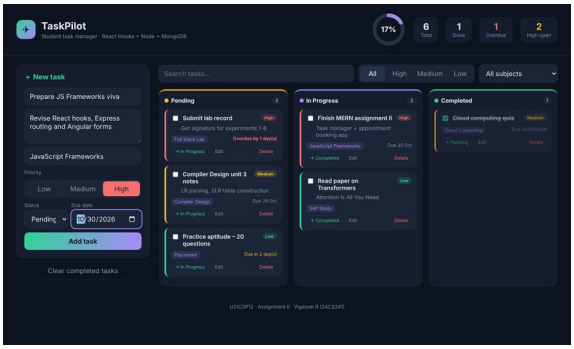


Fig. 6: ADD – new task with High priority and due date

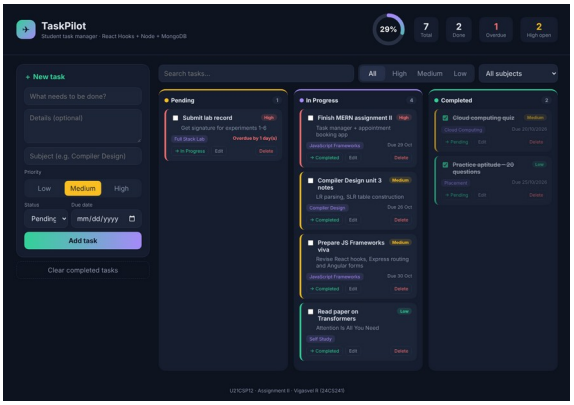


Fig. 7: Completion status changed (moved / ticked)

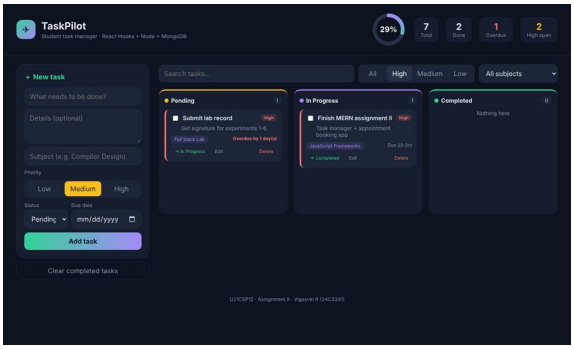


Fig. 8: Filter by priority = High

Result: The application was implemented and all required operations were tested successfully.

Q3 (Task 2) – MediSlot – Appointment Booking (MEAN)

Task: *Design an appointment booking application: create Angular forms for patient registration, display doctors and their available appointment slots, develop REST APIs for appointment management, store patient and appointment records in MongoDB.*

Stack: Angular 20 (standalone components, Router, Reactive Forms, signals) · Node.js + Express.js · MongoDB (vigasvel_medislot)

Requirements and Implementation

Requirement	Implementation
Angular forms for patient registration	PatientRegister page – Reactive Form with required, min/max, pattern, email and consent validators
Display doctors and their available slots	BookAppointment page – doctor cards by specialization and per-date slot chips (free / booked)
REST APIs for appointment management	POST / GET / PATCH / DELETE /api/appointments plus doctors and patients APIs
Store patient and appointment records in MongoDB	Patient, Doctor and Appointment collections; auto Patient ID and daily token number

Database

Doctor: name, specialization, qualification, experience, room, fee, days[], slots[] | Patient: patientId (auto MS-1001…), name, age, gender, phone, email, bloodGroup, address | Appointment: patient (ref), doctor (ref), date, time, reason, status Booked / Cancelled / Completed, token

REST API

Method	Endpoint	Purpose
GET	`/api/doctors`	List doctors (query: specialization)
GET	`/api/doctors/:id/slots?date=`	Slots for a date with available / booked flags
POST	`/api/patients`	Register a patient (generates Patient ID)
GET	`/api/patients/:patientId`	Find a patient
POST	`/api/appointments`	Book a slot (blocks double booking – 409)
GET	`/api/appointments`	List (query: patientId, date, status)
PATCH	`/api/appointments/:id/status`	Cancel or complete an appointment
DELETE	`/api/appointments/:id`	Delete an appointment

Output

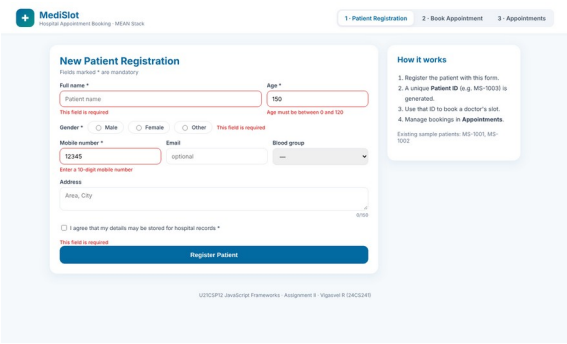


Fig. 9: Angular form validation – patient registration

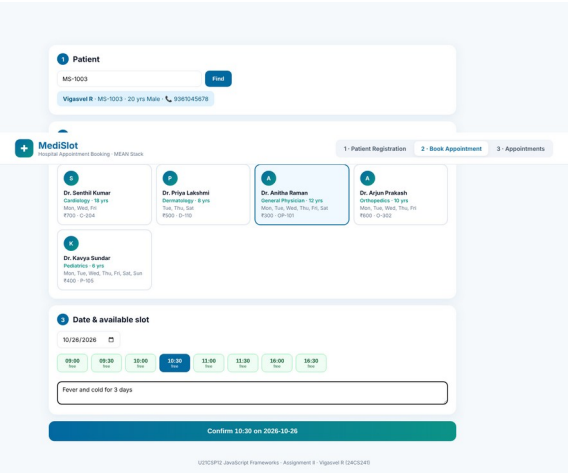


Fig. 10: Doctors and available slots for the chosen date

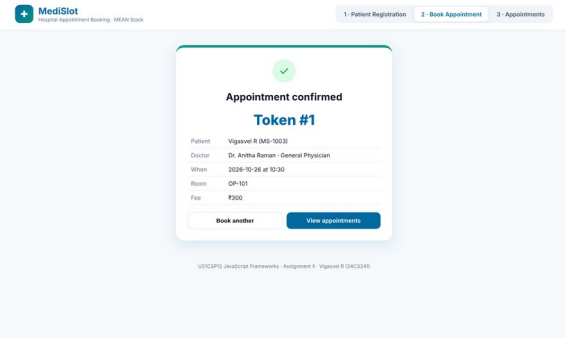


Fig. 11: Appointment confirmed with token number

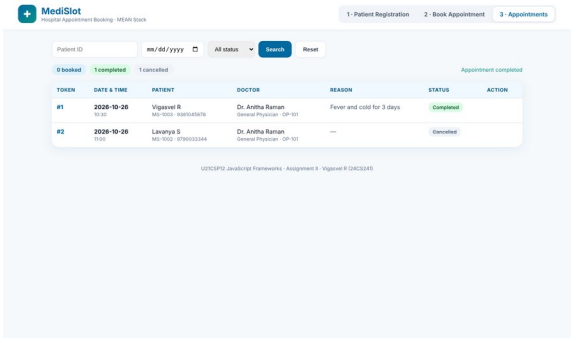


Fig. 12: Appointments completed / cancelled

Result: The application was implemented and all required operations were tested successfully.

Conclusion: All three questions were implemented as working MERN/MEAN applications using React Hooks, Angular forms and services, Express REST APIs and MongoDB.